19. SDK 구조 및 개발자 도구 분석

개요

Omni SDK는 Omni SolverNet과 상호작용하기 위한 TypeScript/JavaScript 라이브러리 모음입니다. 현재 알파 버전으로 React 훅을 중심으로 구성되어 있으며, 향후 다른 프레임워크 지원과 저수준 API를 추가할 예정입니다.

SDK 아키텍처

워크스페이스 구조

```
sdk/
├── packages/ # 핵심 패키지들
| ├── core/ # 핵심 로직 (@omni-network/core)
| ├── react/ # React 훅 (@omni-network/react)
| ├── svm/ # Solana VM 지원 (@omni-network/svm)
| └── test-utils/ # 테스트 유틸리티
├── examples/ # 사용 예제
| ├── core/ # 핵심 라이브러리 예제
| └── react/ # React 예제
├── integration-tests/ # 통합 테스트
└── scripts/ # 빌드 스크립트
```

패키지 관리

```
// sdk/package.json
{
"name": "omni-sdk",
"private": true,
"packageManager": "pnpm@9.12.1",
"scripts": {
"build": "pnpm run --r --filter \"./packages/**\" build",
"changeset:publish": "pnpm changeset:prepublish && changeset publish",
"test:unit": "pnpm run --filter '@omni-network/*' test",
"test:integration": "cd integration-tests && pnpm test"
}
}
```

주요 특징:

- PNPM 워크스페이스 사용
- Changesets을 통한 버전 관리
- Biome을 통한 코드 포맷팅 및 린팅
- TypeScript 기반 개발

핵심 패키지 분석

1. Core 패키지 (@omni-network/core)

역할: Omni SolverNet과의 상호작용을 위한 핵심 로직

```
// sdk/packages/core/package.json
{
"name": "@omni-network/core",
"version": "0.3.2",
"dependencies": {
"viem": "catalog:",
"zod": "^3.25.7"
}
}
```

1.1 API 모듈

견적 조회 (getQuote):

```
// sdk/packages/core/src/api/getQuote.ts
export type GetQuoteParameters = {
srcChainId?: number
destChainId: number
environment?: Environment | string
} & (
| {
mode: 'deposit'
deposit?: Omit<Quoteable, 'amount'>
```

```
expense: Omit<Quoteable, 'amount'> & { amount: bigint }
}
| {
mode: 'expense'
deposit: Omit<Quoteable, 'amount'> & { amount: bigint }
expense?: Omit<Quoteable, 'amount'>
}
)


export async function getQuote(quote: GetQuoteParameters): Promise<Quote> {
const apiUrl = getApiUrl(environment)
const json = await fetchJSON(`${apiUrl}/quote`, {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: toJSON({
sourceChainId: srcChainId,
destChainId: destChainId,
deposit: depositInput ? toQuoteUnit(depositInput, mode === 'deposit') : {},
expense: expenseInput ? toQuoteUnit(expenseInput, mode === 'expense') : {},
}),
})
return {
deposit: { ...deposit, amount: fromHex(deposit.amount as Hex, 'bigint') },
```

```
expense: { ...expense, amount: fromHex(expense.amount as Hex, 'bigint') },
}
}
```

자산 조회 (getAssets):

```
// 지원되는 토큰과 체인 정보 조회
export async function getAssets(environment?: Environment): Promise<Asset[]>
```

컨트랙트 조회 (getContracts):

```
// Omni 컨트랙트 주소 정보 조회
export async function getContracts(environment?: Environment): Promise<OmniCo
```

1.2 컨트랙트 상호작용

주문 전송 (sendOrder):

```
// sdk/packages/core/src/contracts/sendOrder.ts
export type SendOrderParams = {
client: WalletClient
inboxAddress: Address
order: EVMOrder
}
```

```
export  async  function  sendOrder(params: SendOrderParams): Promise<Hex> {
const { client,  inboxAddress,  order } =  params
return  await  client.writeContract({
address: inboxAddress,
abi: inboxAbi,
functionName:  'open',
args: [order],
value: order.deposit.amount,
})
}
```

주문 상태 확인:

```
// 주문 완료 여부 확인
export  async  function  didFill(params: DidFillParams): Promise<boolean>

// 주문 정보 조회
export  async  function  getOrder(params: GetOrderParams): Promise<Order | null>

// 거부 사유 조회
export  async  function  getRejection(params: GetRejectionParams): Promise<Rejec
```

1.3 플로우 관리

주문 생성 플로우:

```
// sdk/packages/core/src/flows/generateOrder.ts
export type GenerateOrderParams = {
srcChainId: number
destChainId: number
deposit: Quoteable
expense: Quoteable
to: Address
data?: Hex
}

export async function generateOrder(params: GenerateOrderParams): Promise<EVM
// 주문 구조체 생성 로직
}
```

주문 오픈 플로우:

```
// sdk/packages/core/src/flows/openOrder.ts
export async function openOrder(params: OpenOrderParams): Promise<OpenOrderRe
// 1. 주문 검증
// 2. 트랜잭션 전송
// 3. 영수증 대기
// 4. 이벤트 파싱
```

```
}
```

1.4 타입 시스템

주문 타입:

```
// sdk/packages/core/src/types/order.ts
export type EVMOrder<abis extends OptionalAbis = {}> = {
srcChainId: number
destChainId: number
deposit: {
token: Address
amount: bigint
}
expense: {
token: Address
amount: bigint
}
to: Address
data: Hex
// ... 기타 필드들
}

export type OrderStatus =
| 'not-found'
```

```
| 'open'
| 'filled'
| 'closed'
| 'rejected'
| 'error'
```

견적 타입:

```
// sdk/packages/core/src/types/quote.ts
export type Quote = {
deposit: {
token: Address
amount: bigint
}
expense: {
token: Address
amount: bigint
}
}

export type Quoteable = {
token?: Address
amount?: bigint
```

```
}
```

2. React 패키지 (@omni-network/react)

역할: React 애플리케이션을 위한 훅 기반 인터페이스

```
// sdk/packages/react/package.json
{
"name": "@omni-network/react",
"version": "0.3.3",
"peerDependencies": {
"@tanstack/react-query": "^5.64.2",
"react": ">=18",
"viem": "2.x",
"wagmi": "2.x"
}
}
```

2.1 컨텍스트 프로바이더

```
// sdk/packages/react/src/context/omni.tsx
type OmniProviderProps = {
env: Environment
__apiBaseUrl?: string
}
```

```
export function OmniProvider({ env, children, __apiBaseUrl }: React.PropsWithC
const apiBaseUrl = __apiBaseUrl ?? getApiUrl(env)
const config = useMemo(() => ({ apiBaseUrl, env }), [env, apiBaseUrl])
const queryClient = useQueryClient()
useEffect(() => {
queryClient.prefetchQuery(getOmniContractsQueryOptions(config))
}, [queryClient, config])

return <OmniContext.Provider value={config}>{children}</OmniContext.Provider>
}
```

2.2 핵심 훅들

주문 관리 훅 (useOrder):

```
// sdk/packages/react/src/hooks/useOrder.ts
type UseOrderParams<abis extends OptionalAbis> = EVMOrder<abis> & {
validateEnabled: boolean
debugValidation?: boolean
omniContractsQueryOpts?: QueryOpts<OmniContracts>
didFillQueryOpts?: QueryOpts<boolean>
rejectionQueryOpts?: QueryOpts<Rejection, GetRejectionError>
}
```

```
type UseOrderReturnType = {
open: UseMutateFunction<`0x${string}`, MutationError, void, unknown>
orderId?: Hex
validation?: UseValidateOrderResult
txHash?: Hex
destTxHash?: Hex
unwatchDestTx?:  ()  => void
error?: UseOrderError
status: UseOrderStatus
isTxPending: boolean
isTxSubmitted: boolean
isValidated: boolean
isOpen: boolean
isError: boolean
isReady: boolean
txMutation: MutationResult
waitForTx: UseWaitForTransactionReceiptReturnType<Config, number>
rejection?: Rejection
}


export  function  useOrder<abis extends OptionalAbis>(params: UseOrderParams<abi
// 1. 컨트랙트 로드
```

```
const contractsResult = useOmniContracts({ queryOpts: omniContractsQueryOpts
// 2. 트랜잭션 뮤테이션
const txMutation = useMutation<SendOrderReturn, MutationError>({
mutationFn: async () => {
const client = await getConnectorClient(config, { chainId: order.srcChainI
return await sendOrder({ client, inboxAddress, order })
},
})
// 3. 트랜잭션 영수증 대기
const wait = useWaitForTransactionReceipt({
hash: txMutation.data,
chainId: order.srcChainId,
})
// 4. 주문 상태 추적
const orderStatus = useGetOrderStatus({
srcChainId: order.srcChainId,
destChainId: order.destChainId,
resolvedOrder,
didFillQueryOpts,
})
// 5. 상태 파생
const status = deriveStatus(contractsResult, orderStatus.status, txMutation
```

```
}
```

견적 조회 훅 (useQuote):

```
// sdk/packages/react/src/hooks/useQuote.ts
export type UseQuoteParams = GetQuoteParameters & {
enabled: boolean
queryOpts?: QueryOpts<Quote, QuoteError>
}

export function useQuote(params: UseQuoteParams): UseQuoteResult {
const { apiBaseUrl } = useOmniContext()
const { enabled, ...quoteParams } = params
const query = useQuery<Quote, QuoteError>({
retry: false,
...params.queryOpts,
queryKey: ['quote', quoteParams],
queryFn: async () => getQuote({ ...quoteParams, environment: apiBaseUrl }),
queryKeyHashFn: hashFn,
enabled,
})

return useResult(query)
```

```
}
```

자산 조회 훅 (useOmniAssets):

```
// 지원되는 자산 목록 조회
export function useOmniAssets(params?: UseOmniAssetsParams): UseOmniAssetsResu
```

컨트랙트 조회 훅 (useOmniContracts):

```
// Omni 컨트랙트 주소 조회
export function useOmniContracts(params?: UseOmniContractsParams): UseOmniCont
```

2.3 상태 관리

주문 상태 파생:

```
function deriveStatus(
contracts: UseOmniContractsResult,
orderStatus: OrderStatus,
txStatus: UseWriteContractReturnType['status'],
receiptStatus: UseWaitForTransactionReceiptReturnType['status'],
receiptFetchStatus: UseWaitForTransactionReceiptReturnType['fetchStatus'],
): UseOrderStatus {
// 컨트랙트 로딩 상태 확인
if (contracts.isError) return 'error'
```

```
if (!contracts.data) return 'initializing'

// 온체인 상태 우선순위
if (orderStatus === 'filled') return 'filled'
if (orderStatus === 'open') return 'open'
if (orderStatus === 'rejected') return 'rejected'
if (orderStatus === 'closed') return 'closed'
if (orderStatus === 'error') return 'error'

// 영수증 상태 우선순위
if (receiptStatus === 'error') return 'error'
if (receiptStatus === 'success') return 'open'
if (receiptFetchStatus === 'idle') return 'ready'
if (receiptStatus === 'pending') return 'opening'

// 트랜잭션 상태 폴백
if (txStatus === 'error') return 'error'
if (txStatus === 'pending') return 'opening'
if (txStatus === 'success') return 'opening'

return 'ready'
}
```

3. SVM 패키지 (@omni-network/svm)

역할: Solana Virtual Machine 지원 (개발 중)

```
// sdk/packages/svm/package.json
{
"name": "@omni-network/svm",
"private": true,
"version": "0.0.1",
"dependencies": {
"@solana-program/token": "^0.5.1",
"@solana/kit": "^2.1.1",
"bs58": "^6.0.0"
}
}
```

주요 기능:

- Solana 프로그램 인터페이스
- Codama 클라이언트 생성
- 토큰 프로그램 통합

4. 테스트 유틸리티 (@omni-network/test-utils)

역할: SDK 패키지들을 위한 공유 테스트 유틸리티

```
// sdk/packages/test-utils/package.json
```

```
{
"name": "@omni-network/test-utils",
"private": true,
"dependencies": {
"@omni-network/core": "workspace:*",
"viem": "catalog:"
}
}
```

개발자 도구

1. 빌드 시스템

TypeScript 설정:

```
// sdk/tsconfig.base.json
{
"compilerOptions": {
"strict": true,
"noFallthroughCasesInSwitch": true,
"noImplicitReturns": true,
"noImplicitOverride": true,
"noUnusedLocals": true,
"noUnusedParameters": true,
```

```
"jsx": "react-jsx",
"moduleResolution": "nodenext",
"module": "nodenext",
"target": "ES2021",
"lib": ["ES2022", "DOM"]
}
}
```

빌드 스크립트:

```
// 각 패키지의 공통 빌드 스크립트
{
"scripts": {
"build": "pnpm clean && pnpm build:ts",
"build:ts": "tsc -p tsconfig.build.json",
"clean": "rm -rf dist tsconfig.tsbuildinfo",
"test": "vitest run"
}
}
```

2. 코드 품질 도구

Biome 설정:

```
// sdk/biome.json
```

```
{
// 코드 포맷팅 및 린팅 규칙
}
```

테스트 프레임워크:

- **Vitest**: 단위 테스트 및 통합 테스트
- **Testing Library**: React 컴포넌트 테스트
- **Happy DOM**: 브라우저 환경 시뮬레이션

3. 버전 관리

Changesets 워크플로우:

```
// sdk/package.json
{
"scripts": {
"changeset:version": "changeset version && pnpm version:bump && pnpm check",
"changeset:publish": "pnpm changeset:prepublish && changeset publish",
"version:bump": "node --experimental-strip-types scripts/bumpVersion.ts"
}
}
```

사용 예제

1. React 애플리케이션 설정

```
// App.tsx
import { OmniProvider } from  '@omni-network/react'
import { QueryClient, QueryClientProvider } from  '@tanstack/react-query'
import { WagmiProvider } from  'wagmi'

const  queryClient  =  new  QueryClient()

function  App() {
return (
<WagmiProvider  config={wagmiConfig}>
<QueryClientProvider  client={queryClient}>
<OmniProvider  env="testnet">
<MyComponent />
</OmniProvider>
</QueryClientProvider>
</WagmiProvider>
)
}
```

2. 견적 조회 사용법

```
// QuoteComponent.tsx
import { useQuote } from  '@omni-network/react'
```

```
function QuoteComponent() {
const quote = useQuote({
mode: 'expense',
srcChainId: 1, // Ethereum
destChainId: 137, // Polygon
deposit: {
token: '0xA0b86a33E6441E6C7D3E4081f7567C8b8b2C7A1B', // USDC
amount: parseUnits('100', 6), // 100 USDC
},
expense: {
token: '0x2791Bca1f2de4661ED88A30C99A7a9449Aa84174', // USDC on Polygon
},
enabled: true,
})

if (quote.isPending) return <div>Loading quote...</div>
if (quote.isError) return <div>Error: {quote.error.message}</div>

return (
<div>
<p>Deposit: {formatUnits(quote.deposit.amount, 6)} USDC</p>
<p>Receive: {formatUnits(quote.expense.amount, 6)} USDC</p>
```

```
</div>
)
}
```

3. 주문 실행 사용법

```
// OrderComponent.tsx
import { useOrder } from '@omni-network/react'

function OrderComponent() {
const order = useOrder({
srcChainId: 1,
destChainId: 137,
deposit: {
token: '0xA0b86a33E6441E6C7D3E4081f7567C8b8b2C7A1B',
amount: parseUnits('100', 6),
},
expense: {
token: '0x2791Bca1f2de4661ED88A30C99A7a9449Aa84174',
amount: parseUnits('99', 6),
},
to: '0x742d35Cc6634C0532925a3b8D4C9db96C4b4d8b6',
data: '0x',
```

```
validateEnabled:  true,
})

const  handleSubmit  =  ()  =>  {
if (order.isReady  &&  order.isValidated) {
order.open()
}
}

return (
<div>
<p>Status: {order.status}</p>
{order.txHash && <p>TX  Hash: {order.txHash}</p>}
{order.destTxHash && <p>Dest  TX  Hash: {order.destTxHash}</p>}
<button
onClick={handleSubmit}
disabled={!order.isReady || !order.isValidated || order.isTxPending}
>
{order.isTxPending ? 'Submitting...' : 'Submit Order'}
</button>
{order.error && <p>Error: {order.error.message}</p>}
</div>
)
```

```
}
```

4. 핵심 라이브러리 직접 사용

```
// core-example.ts
import { getQuote, generateOrder, sendOrder } from '@omni-network/core'
import { createWalletClient, http } from 'viem'

async function executeOrder() {
// 1. 견적 조회
const quote = await getQuote({
mode: 'expense',
srcChainId: 1,
destChainId: 137,
deposit: {
token: '0xA0b86a33E6441E6C7D3E4081f7567C8b8b2C7A1B',
amount: parseUnits('100', 6),
},
expense: {
token: '0x2791Bca1f2de4661ED88A30C99A7a9449Aa84174',
},
environment: 'testnet',
})
```

```
// 2. 주문 생성
const order = await generateOrder({
srcChainId: 1,
destChainId: 137,
deposit: quote.deposit,
expense: quote.expense,
to: '0x742d35Cc6634C0532925a3b8D4C9db96C4b4d8b6',
data: '0x',
})

// 3. 주문 전송
const client = createWalletClient({
transport: http('https://eth-mainnet.g.alchemy.com/v2/...'),
})

const txHash = await sendOrder({
client,
inboxAddress: '0x...',
order,
})

console.log('Order submitted:', txHash)
```

```
}
```

통합 테스트

테스트 구조

```
// sdk/integration-tests/package.json
{
"dependencies": {
"@omni-network/core": "workspace:*",
"@omni-network/react": "workspace:*",
"@tanstack/react-query": "catalog:",
"ethers": "^6.11.1",
"react": "catalog:",
"viem": "catalog:",
"wagmi": "catalog:"
}
}
```

테스트 범위:

- API 엔드포인트 통합 테스트
- React 훅 동작 테스트
- 크로스체인 시나리오 테스트

- 오류 처리 테스트

개발 워크플로우

1. 로컬 개발

```
# 의존성 설치
pnpm install

# 모든 패키지 빌드
pnpm build

# React 예제 실행
pnpm dev:react

# 테스트 실행
pnpm test:unit
pnpm test:integration
```

2. 패키지 배포

```
# 변경사항 추가
pnpm changeset
```

```
# 버전 업데이트
pnpm changeset:version

# 패키지 배포
pnpm changeset:publish
```

3. 코드 품질 관리

```
# 코드 포맷팅 및 린팅
pnpm check

# 타입 체크
pnpm build:ts
```

환경 설정

지원 환경

```
// sdk/packages/core/src/types/config.ts
export type Environment = 'mainnet' | 'testnet' | 'devnet'

export type OmniConfig = {
apiBaseUrl: string
env: Environment
```

```
}
```

API 엔드포인트

```
// 환경별 API URL 설정
export function getApiUrl(environment?: Environment | string): string {
switch (environment) {
case 'mainnet':
return 'https://api.omni.network'
case 'testnet':
return 'https://api.testnet.omni.network'
case 'devnet':
return 'http://localhost:8080'
default:
return 'https://api.testnet.omni.network'
}
}
```

향후 계획

1. 프레임워크 확장

- Vue.js 지원
- Angular 지원

- Vanilla JavaScript 지원

2. 기능 확장

- 고급 주문 타입 지원
- 배치 주문 처리
- 실시간 상태 업데이트

3. 개발자 경험 개선

- 더 나은 타입 안전성
- 향상된 문서화
- 더 많은 예제 제공

결론

Omni SDK는 크로스체인 애플리케이션 개발을 위한 포괄적인 도구 세트를 제공합니다. 주요 특징은:

1. **모듈화된 아키텍처**: 핵심 로직과 프레임워크별 구현 분리
2. **타입 안전성**: TypeScript 기반의 강력한 타입 시스템
3. **React 통합**: 훅 기반의 직관적인 API
4. **개발자 친화적**: 풍부한 예제와 테스트 코드
5. **확장 가능성**: 새로운 체인과 기능 추가 용이

이러한 특징들을 통해 개발자들이 Omni SolverNet을 활용한 크로스체인 애플리케이션을 쉽게 구축할 수 있도록 지원합니다.